

INFORME PRELIMINAR

INFORMÁTICA II

SEBASTIAN VERA SANCHEZ

1022036275

DIEGO ALEJANDRO COROBA DOMINGUEZ

1036653212

UNIVERSIDAD DE ANTIOQUIA

FACULTAD DE INGENIERÍA

INGENIERÍA EN TELECOMUNICACIONES

MEDELLÍN

2026

En el problema, se requiere la elaboración de un juego similar al popular Candy Crush, en C++, con varias restricciones de librerías, el programa debe ser diseñado por medio de manipulación de bits de sus operaciones y memoria dinámica en donde se debe crear un tablero de forma dinámica, eliminación de fichas, codificar la estructura del tablero, eliminación de filas y de columnas . Todo esto mediante la manipulación de datos con punteros, memoria dinámica y operadores de bit.

El tablero se debe representar por medio de arreglos continuos , donde cada casilla tiene un valor de 3 bits lo que da un 8 combinaciones o estados, las casillas del tablero están almacenadas en secuencias continuas esto por medio de puntero, sus dimensiones serán de de (F)filas y (C)Columnas, uno de los retos es poder determinar la cantidad mínima de bytes necesarios para almacenar el tamaño del tablero en la memoria.

Para empezar decidimos crear un repositorio con el nombre de desafio-1 y dentro de ese repositorio creamos dos dos branches con el fin de no interrumpir el trabajo del otro y no generar errores de sincronización.

The screenshot shows the GitHub interface for the repository 'desafio-1' by user 'sebastianveras-netizen'. The 'Branches' tab is selected, showing a list of branches. The 'main' branch is the default. Two other branches, 'diego' and 'sebastian', are listed under the 'Your branches' and 'Active branches' sections. Each branch entry shows its update status, check status, and whether it is behind or ahead of the default branch.

Branch	Updated	Check status	Behind	Ahead	Pull request
main	3 days ago				Default
diego	3 days ago		0	0	
sebastian	3 days ago		0	1	

planteandonos diferentes problemáticas como la implementación de una función que nos pase los bits a letras teniendo este como función principal discriminar las letras por cada 3 bits pensamos utilizar el operador de bit &. ejm: si a= verdadero y b= verdadero entonces condición 1= a & b; ejm 7 en binario de 8 bits lo utilizamos para limpiar y obtener solo 3 bits significativos 2 (00000010)

& 7 (00000111)
00000010

siendo este como resultado 2 y pudiendo asignar el valor de C a este por medio de la suma de bits + carácter ejm 'A' + 0 = 65 —> 'A'

'A' + 1 = 66 —> 'B'

'A' + 2 = 67 —> 'C'

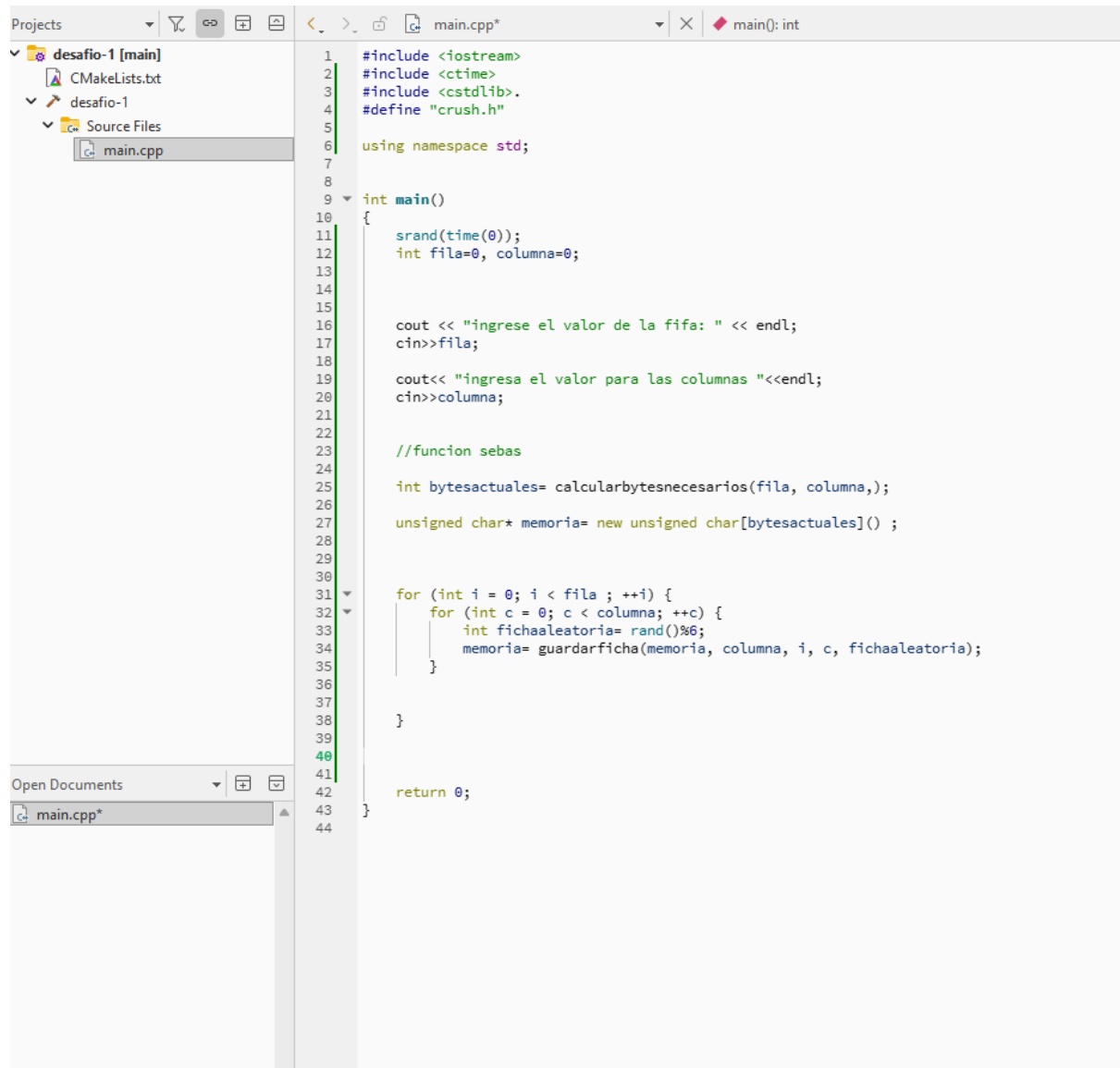
'A' + 3 = 68 —> 'D'

'A' + 4 = 69 —> 'E'

'A' + 5 = 70 —> 'F'

ahora ya sabemos como vamos a convertir los bits en caracteres nos falta verificar cómo realizamos la distribución de los bytes de tal forma que un char de 1 byte nos alcanza para 2 caracteres y nos sobran 2 bits pudiendo utilizarlos para una hilera más de bytes si añadimos más o si el programa lo necesita pues todo depende de la cantidad de filas y columnas que asignemos ya que este valor de bytes necesario se lograria así filas x columnas x 7 y si sobran bytes va a requerir ajustarlo para dar el último, siendo necesario sumar 7 ejm 6 filas x 6 columnas x 3 bits de cada ficha + 7 = 115 y verificamos cuantos bytes requerimos dividiendo entre 8 qué es la cantidad de bits que tiene cada byte 115/8=14 despreciamos las décimas y nos quedamos con el valor entero pudiendo asignar este valor a un arreglo. para determinar el tamaño de memoria que requiere el tablero y poder efectuar las reducciones respectivas que nos piden más adelante.

main



```
1 #include <iostream>
2 #include <ctime>
3 #include <cstdlib>.
4 #define "crush.h"
5
6 using namespace std;
7
8
9 int main()
10 {
11     srand(time(0));
12     int fila=0, columna=0;
13
14
15     cout << "ingrese el valor de la fifa: " << endl;
16     cin>>fila;
17
18     cout<< "ingresa el valor para las columnas "<<endl;
19     cin>>columna;
20
21
22     //funcion sebas
23
24     int bytesactuales= calcularbytesnecesarios(fila, columna,);
25
26     unsigned char* memoria= new unsigned char[bytesactuales]() ;
27
28
29
30
31     for (int i = 0; i < fila ; ++i) {
32         for (int c = 0; c < columna; ++c) {
33             int fichaaaleatoria= rand()%6;
34             memoria= guardarficha(memoria, columna, i, c, fichaaaleatoria);
35         }
36     }
37
38
39
40
41     return 0;
42 }
43
44
```

iniciamos el programa incluyendo dos librerías `<ctime>` y el `<cstdlib>` las cuales fueron necesarias para poder crear una variable aleatoria, esta usa la función **srand** como el punto de partida de la semilla y dentro de esa función usamos el **time** que viene de la librería `<ctime>` en esta se están generando un contador por segundo el cual el pieza clave en el código ya que la función **srand** necesita un valor para que se pueda ejecutar. Luego de dar inicio a la semilla, damos un par de instrucciones por medio de la consola solicitando dos valores enteros y asignarlas a las variables **fila** y las **columna**. el valor de estas variables irán a la función **bytesactuales** y me retorna un valor en bits este valor dependerá de los valores que ingresen en filas y columnas. al final este variable entra en un puntero de tipo `unsigned char` y de nombre memoria en cual me reserva un espacio de memoria en el heap, con un arreglo que es el resultado de la función **bytesactuales**. el siguiente fragmento de nuestro código es una interacción de dos ciclos `for` y estas se encargan de hacer una interacción de cada fila y cada columna, el primer ciclo `for` hace un recorrido de las filas y el segundo a las columnas, en cada interacción de fila y columna se

modifica el valor de la variable *ficha* aleatoria el cual toma el valor de la semilla y se le saca el modulo 6 este ultimo me indica que manejaremos un rango de 0 a 5.

al final de cada interacción los resultados van hacia una función que definimos como guardar *ficha* y esta va a esta alojada dentro de nuestro puntero que llamamos memoria

```
int calcularbytesnecesarios (int filas , int columnas) {  
    int totalbits = filas * columnas * 3;  
    return (totalbits + 7) / 8;  
}
```

Creamos la función *calcularbytesnecesarios* con la intención de saber cuántos bytes serían requeridos, y así poder asignar una cantidad exacta en el arreglo que nos dará espacio en la memoria dinámica (exactamente en el *heap*). Esto se hace con la intención de poder asignarle una posición de memoria exacta a cada bit seleccionado para formar parte de nuestros caracteres.

Así, este puede ser asignado al byte #0, 1 o 2... en los bits del 0 al 2 o del 3 al 5; o pudiendo empezar en el bit 6 y terminar en el siguiente byte, tomando un bit de este y dándonos un nuevo inicio en el bit 1 y terminando en el 3, luego del 4 al 6, y dejándonos 1 bit huérfano que podrá ser completado con el residuo del otro byte. Teniendo presente que cada byte tiene 8 bits, por ende, siempre sobran 2, y seguiremos realizando esta operación para poder asignarle un lugar a cada conjunto de 3 bits en el *heap*.

```

}
unsigned char* guardarfichas ( unsigned char* memoria,int columnas, int fila,int columna, int valor){
    valor = valor & 7 ;
    int indiceficha = fila * columnas + columna;
    int bitinicial = indiceficha * 3;
    int numerobytes = bitinicial/8;
    int posicionenbyte = bitinicial % 8;

    if (posicionenbyte <= 5) {
        memoria[numerobytes] &= ~(7 << posicionenbyte);
        memoria[numerobytes] |= (valor << posicionenbyte);
    }

    else if (posicionenbyte == 6) {
        int parte1 = valor & 3;
        int parte2 = (valor >> 2) & 1;
        memoria[numerobytes] &= ~(3 << 6);
        memoria[numerobytes] |= (parte1 << 6);

        memoria[numerobytes + 1] &= ~1;
        memoria[numerobytes + 1] |= parte2;
    }

    else {
        int parte1 = valor & 1;
        int parte2 = (valor >> 1) & 3;

        memoria[numerobytes] &= ~(1 << 7);
        memoria[numerobytes] |= (parte1 << 7);

        memoria[numerobytes + 1] &= ~3;
        memoria[numerobytes + 1] |= parte2;
    }

    return memoria;
}

```

declaramos la función guardar fichas con un puntero apuntando a la posición de guardado en la memoria con el puntero memoria para acceder directamente a la heap asignada para

el tablero siendo este la cantidad de bytes que se asignaron para el tablero anteriormente por medio de un arreglo luego se selecciona en qué posición vamos a empezar a asignar caracteres utilizando la cantidad de columnas que selecciono el usuario nos desplazamos por este puntero y seleccionamos la fila y la columna siendo asignada esta, de uno en uno gracias a un ciclo for que efectuamos anteriormente en el main para así ir avanzando la posición de uno en uno luego asignamos un número desde main para la variable valor la cual se produce mediante `rand ( )` módulo 6 teniendo estos 5 índices empezamos a operar empezando por recortar a (valor) por medio de una máscara aplicada al 7 el cual nos asigna valores para los 3 primeros bits dejándonos limpios los bits contiguos luego verificamos en qué posición vamos a asignarlo con (`indiceficha = fila x columnas + columna`) ejemplo `fila=0 columnas=3 y columna=0`)
`indiceficha = 0`

`bitinicial=indiceficha x 3 = 0`

`numerobytes=bitinicial/8` lo dividimos entre 8 para poder calcular la cantidad de bytes necesarias como bitinicial nos da 0 entonces `0/8=0`

`posicionenbyte = bitinicial % 8` nos da la posición inicial desde la cual se dará el guardado de los 3 bits asignados a cada carácter. como bit inicial es 0 entonces `0 % 8` es 0 dándonos la posición inicial para empezar el guardado de los bits en el byte 0

ahora verificamos como se va a guardar este entonces si `posicionenbyte` es 0 (`0<=5`)

si es verdadero entonces entramos empezamos por memoria en la posición 0

hacemos un desplazamiento de 0 posiciones para la máscara 7 hacia la izquierda

y limpiamos los bits que vamos a utilizar con `~` ya que la función de este es invertir cada bit si es un 1 se convierte en 0 o si es 0 lo convierte en 1 siendo así la máscara 7 (00000111) al aplicar `~` queda (11111000) pudiendo asignar un valor a estos 3 primeros bits

entonces memoria en la posición 0 se le añade con (`|`) el número en bits que tenga la variable (valor) y nos desplazamos hacia la izquierda la cantidad de `posicionenbyte` como la `posicionenbyte` es 0 no nos desplazamos y queda asignado para este puesto el número que nos dio la variable valor

por último retornamos memoria quedando así guardado el número que fue asignado en valor que posteriormente será convertido en un carácter

tenemos los diferentes casos de cuando nos da más de 5 pues dependiendo de esto nos vemos obligados a utilizar los bits de los otros bytes y a correrlos dentro de estos para asignar los bits pertinentes que nos haya dado la semilla a la variable (valor) aplicando anteriormente módulo 6 a cada semilla asegurándonos de estar en el rango de las 6 letras que queremos para esto siendo 0=A , 1=B , 2=C , 3=D , 4=E , 5=F.

sweet crush

cuadrícula variable en
tamaño por el
usuario

definición de las
letras en código binario

utilizar todos los
bits de un byte
que tiene un char

Contigüación aleatoria
del orden de las letras

caída en cascada
de las letras
tras eliminar una
letra o varias

tener en cuenta los
bits (3)

calcular almacenamiento
requerido para un punteto
y aplicar arreglos a este
dependiendo de las filas
y columnas asignadas

recopilar cada
información
de los movimientos
y bits o bytes
utilizados